LOCAL VM AUTO-SCALING TO GCP

Platform	Language	Cloud	Threshold
VirtualBox + Ubuntu	Python 3.10+	GCP asia-south1-a	CPU / RAM > 75%

1. Project Overview

This project implements a threshold-triggered cloud-burst mechanism that monitors resource utilisation on a local VirtualBox VM and dynamically provisions a GCP Compute Engine instance when CPU or memory usage exceeds 75%. It demonstrates hybrid-cloud workload management using only a Python script and the GCP REST API - no manual intervention required.

1.1 Key Objectives

- Monitor CPU and RAM in real time using the psutil Python library.
- Trigger GCP VM provisioning the moment either metric exceeds 75%.
- Deploy Apache2 on the new GCP instance via startup script — no SSH required.
- Demonstrate the full end-to-end flow with a simulated stress load.

1.2 Technology Stack

Component	Technology / Tool
Local VM Hypervisor	Oracle VirtualBox 7.x
Guest OS	Ubuntu 22.04 LTS (64-bit) · 2 vCPU · 4 GB RAM
Monitoring Language	Python 3.10+ with psutil 5.9+
Cloud Provider / Service	GCP Compute Engine — e2-micro, asia-south1-a
Cloud API Client	google-api-python-client 2.x + Service Account (JSON key)
Demo Application	Apache2 web server (deployed via startup script)
Load Simulator	stress (Linux apt package)

2. Architecture Design

The architecture follows a three-tier pattern: Local Monitoring → Cloud API Integration → Cloud Provisioning.

Tier	Component	Responsibility
Tier 1 — Monitoring	monitor.py + psutil	Poll CPU% & RAM% every 5 s; evaluate threshold
Tier 2 — API Integration	gcp-key.json + GCP Compute API	Authenticate and POST instances.insert request
Tier 3 — Provisioning	GCP e2-micro + startup-script	Boot Debian 11 VM; install & start Apache2

Flow summary: psutil samples metrics → max(cpu, ram) compared to THRESHOLD → if exceeded, service account credentials loaded → Compute API called → GCP provisions VM → startup script installs Apache2 → public HTTP endpoint live.

3. Implementation

3.1 GCP Setup

- Create a GCP project and enable the Compute Engine API.
- Create a service account (autoscale-sa) with the Compute Admin role.
- Generate a JSON key and save it to ~/monitoring/gcp-key.json inside the VM.
- Never commit gcp-key.json — add it to .gitignore.

3.2 monitor.py — Core Script

Install dependencies:

```
pip3 install psutil google-api-python-client google-auth
```

Key configuration constants at the top of the script:

```
THRESHOLD = 75        # % trigger
PROJECT   = 'your-project-id'
ZONE      = 'asia-south1-a'
INSTANCE_NAME = 'auto-vm-instance'
KEY_FILE  = 'gcp-key.json'
```

get_usage() — metric collection:

```
cpu    = psutil.cpu_percent(interval=1)          # 1s window, avoids 0.0% artefact
memory = psutil.virtual_memory().percent
return max(cpu, memory)                          # scale on worst metric
```

trigger_cloud() — GCP provisioning (called once threshold is exceeded):

```
credentials = service_account.Credentials.from_service_account_file(KEY_FILE)
compute     = discovery.build('compute', 'v1', credentials=credentials)
config = {                                       # VM config dict
  'name': INSTANCE_NAME,
  'machineType': f'zones/{ZONE}/machineTypes/e2-micro',
  'disks': [{'boot':True,'autoDelete':True,
    'initializeParams':{'sourceImage':'projects/debian-cloud/global/images/family/debian-
11'}}],
  'networkInterfaces': [{'network':'global/networks/default',
    'accessConfigs':[{'type':'ONE_TO_ONE_NAT','name':'External NAT'}]}],
  'metadata': {'items':[{'key':'startup-script','value':'#!/bin/bash\napt update\napt install
apache2 -y'}]}
}
compute.instances().insert(project=PROJECT, zone=ZONE, body=config).execute()
```

Main loop:

```
while True:
    usage = get_usage()
    if usage > THRESHOLD:
        trigger_cloud(); break
    time.sleep(5)
```

4. Running the Demo

4.1 Pre-flight Checklist

Item	Requirement

VM	Ubuntu 22.04 booted; Python deps installed
Credentials	gcp-key.json at ~/monitoring/gcp-key.json
Config	PROJECT set to actual GCP project ID in monitor.py
API	Compute Engine API enabled in GCP Console
Stress tool	sudo apt install stress -y completed
Clean state	No existing auto-vm-instance in GCP Console

4.2 Demo Steps

- Terminal 1 — start monitor: cd ~/monitoring && python3 monitor.py
- Terminal 2 — simulate load: stress --cpu 2 --timeout 60
- Monitor detects CPU > 75% within next 5 s poll cycle and triggers GCP API.
- GCP Console: Compute Engine → VM instances → auto-vm-instance (PROVISIONING → RUNNING).
- After ~3 min, open http://<EXTERNAL_IP>/ to see the Apache2 demo page.
- Cleanup: gcloud compute instances delete auto-vm-instance --zone=asia-south1-a

5. Security Considerations

Concern	Guidance
Key file protection	gcp-key.json grants Compute Admin access. Never commit to Git or share. Add to .gitignore.
VM network access	Demo uses public IP (ONE_TO_ONE_NAT). In production, restrict with OS Login and VPC firewall rules.
Principle of least privilege	Compute Admin is broad. In production, create a custom role with only compute.instances.create/delete and compute.zones.list.
Production auth	Replace JSON key file with Workload Identity Federation to eliminate static credentials.

6. Testing & Troubleshooting

6.1 Key Test Scenarios

Scenario	Expected Result
get_usage() at idle	Returns 0–30%; CPU and RAM print correctly
get_usage() during stress --cpu 2	Returns > 75%; scaling triggers
Wrong PROJECT ID	HttpError 404 — project not found
Missing gcp-key.json	FileNotFoundError on credentials load
Duplicate instance name	HttpError 409 — change INSTANCE_NAME and retry
Compute Engine API not enabled	HttpError 403 — enable in GCP Console

6.2 Common Errors & Fixes

Error / Issue	Resolution
ModuleNotFoundError: psutil	pip3 install psutil
ModuleNotFoundError: googleapiclient	pip3 install google-api-python-client google-auth
HttpError 403: IAM permission denied	Verify service account has Compute Admin role
HttpError 400: resource already exists	Delete existing VM or change INSTANCE_NAME
CPU never exceeds 75%	Try stress --cpu 4 --vm 2 or lower THRESHOLD to 50
Apache2 page not loading	Wait 3–5 min after RUNNING; check firewall allows port 80
get_usage() returns 0.0	Use psutil.cpu_percent(interval=1), not interval=0

7. Repository Structure & Conclusion

7.1 Repository Layout

```
autoscale-project/
  src/          monitor.py  ·  startup-script.sh  ·  requirements.txt
  docs/         architecture.svg  ·  architecture.png  ·  report.docx
  demo/         demo-video.mp4
  .gitignore    (excludes gcp-key.json)
  README.md     quick-start guide
```

7.2 Conclusion

This project successfully delivers a lightweight, script-driven hybrid-cloud auto-scaling mechanism. No specialised tooling (Kubernetes, Terraform, paid monitoring) is required — only Python, a GCP service account, and the Compute REST API.

All stated objectives are met: psutil polls metrics continuously, the threshold check triggers within 5 seconds of a CPU/RAM spike, the GCP VM is provisioned without manual intervention, and Apache2 is accessible via public IP within minutes.

The architecture is extensible: the threshold can be tuned, additional metrics added to get_usage(), the machine type upgraded, and the startup script replaced with any provisioning logic. Natural next steps include bi-directional scaling, health-check polling, Terraform templates, and Workload Identity Federation.